

CHAPTER 9

THE PROCESS OF INTERACTION DESIGN

[9.1 Introduction](#)

[9.2 What Is Involved in Interaction Design?](#)

[9.3 Some Practical Issues](#)

Objectives

The main aims of this chapter are to:

- Consider what doing interaction design involves.
- Explain some advantages of involving users in development.
- Explain the main principles of a user-centered approach.
- Present a simple lifecycle model of interaction design.
- Ask and provide answers for some important questions about the interaction design process.
- Consider how interaction design activities can be integrated into the wider product development lifecycle.



9.1 Introduction

Design is a practical and creative activity with the aim of developing a product that helps its users achieve their goals. In previous chapters, we looked at different kinds of interactive products, issues that need to be taken into account when doing interaction design, some of the theoretical bases for the field, and techniques for gathering and analyzing data to understand users' goals. In this chapter we start to explore how we can design and build interactive products.

[Chapter 1](#) defined interaction design as being concerned with 'designing interactive products to support the way people communicate and interact in their everyday and working lives.' But how do you go about doing this? Developing a product must begin with gaining some understanding of what is required of it, but where do these requirements come from? Whom do you ask about them? Underlying good interaction design is the philosophy of user-centered design, i.e. involving users throughout development, but who are the users? Will they know what they want or need even if we can find them to ask? For an innovative product, users are unlikely to be able to envision what is possible, so where do these ideas come from?

In this chapter, we raise and answer these kinds of questions, discuss user-centered design, and revisit the four basic activities of the interaction design process that were introduced in [Chapter 1](#). We also introduce a lifecycle model of interaction design that captures these activities.

9.2 What Is Involved in Interaction Design?

The previous chapters have introduced you to many exciting ideas, approaches, theories, and opinions about interaction design, but what does it mean to actually do interaction design? The following activity is intended to start you thinking about this by asking you to produce an initial design for an interactive product.

Activity 9.1

Imagine that you want to design a travel planner for yourself. You might use this system to plan your route, check visa requirements, book flights or train tickets, investigate the facilities at your destination, and so on. Assume that the system is destined to run on a tablet for the purposes of this activity.

1. Make a list of the user experience and usability goals for the system.
2. Outline the initial screen or two for this system, showing its main functionality and its general look and feel. Spend about 10 minutes on this.
3. Having produced an outline, spend 5 minutes reflecting on how you went about tackling this activity. What did you do first? Did you have any particular artifacts or experience to base your design upon? What process did you go through?

Comment

Show/Hide

BOX 9.1

Four Approaches to Interaction Design

Saffer (2010) suggests four main approaches to interaction design, each of which is based on a distinct underlying philosophy: user-centered design, activity-centered design, systems design, and genius design. He acknowledges that the purest form of any of these is unlikely to be realized, and takes an extreme view of each in order to distinguish between them. In user-centered design, the user knows best and is the only guide to the designer; the designer's role is to translate the users' needs and goals into a design solution. Activity-centered design focuses on the behavior surrounding particular tasks. Users still play a significant role but it is their behavior rather than their goals and needs that are important. Systems design is a structured, rigorous, and holistic design approach that focuses on context and is particularly appropriate for complex problems. In systems design it is the system (i.e. the people, computers, objects, devices, and so on) that are the center of attention while the users' role is to set the goals of the system.

Finally, genius design is different from the other three approaches because it relies solely on the experience and creative flair of a designer. Jim Leftwich, an experienced interaction designer interviewed in Saffer (2010, pp. 44–5) prefers the term 'rapid expert design.' In this approach the users' role is to validate ideas generated by the designer, and users are not involved during the design process itself. Saffer points out that this is not necessarily by choice, but may be due to limited or no resources for user involvement. Apple, for example, does very little user research or testing, yet the Apple iPod is acknowledged as a significant design achievement.

Different design problems lend themselves more easily to different approaches, and different designers will tend to gravitate towards using the approach that suits them best. Although an individual designer may prefer a particular approach, it is important that the approach for any one design problem is chosen with that design problem in mind. ■

There are many fields of design; for example, graphic design, architectural design, industrial and software design, and although each discipline has its own interpretation of 'designing,' there are three fundamental activities that are recognized in all design: understanding the requirements, producing a design that satisfies those requirements, and evaluating the design. Interaction design also involves these activities, and in addition we focus attention very clearly on users and their goals. For example, we investigate the artifact's use and target domain by taking a user-centered approach to development, we seek users' opinions and reactions to early designs, and we involve users appropriately in the development process itself. This means that users' concerns direct the development rather than just technical concerns, and for interaction design, the three fundamental activities of design listed above are extended to include an activity of prototyping so that users can interact with the design.

So design involves work on requirements, designing a solution, producing an interactive version of the solution, and evaluating it. But design is also about trade-offs, about balancing conflicting requirements. One common form of trade-off when developing a system to offer advice is deciding how much choice will be given to the user and how much direction the system should offer. Often the division will depend on the purpose of the system, e.g. for business travel or for vacations. Getting the balance right requires experience, but it also requires the development and evaluation of alternative solutions.

Generating alternatives is a key principle in most design disciplines, and one that should be encouraged in interaction design. Linus Pauling, twice a Nobel Prize winner, once said: "The best way to get

a good idea, is to get lots of ideas.” This is not necessarily easy, however, and unlike many design disciplines, interaction designers are not generally trained to generate alternative designs. The good news is that the ability to brainstorm and contribute alternative ideas can be learned. For example, Kelley (2008) describes seven secrets for better brainstorms, including sharpen the focus (have a well-honed problem statement), playful rules (to encourage ideas), and get physical (use visual props). He also discusses six ways to kill a brainstorm, including do it off-site and write everything down.

Involving users and others in the design process means that the designs and potential solutions will need to be communicated to people other than the original designer. This requires the design to be captured and expressed in some suitable form that allows review, revision, and improvement. There are many ways of doing this, one of the simplest being to produce a series of sketches. Other common approaches are to write a description in natural language, to draw a series of diagrams, and to build prototypes. A combination of these techniques is likely to be the most effective. When users are involved, capturing and expressing a design in a suitable format is especially important since they are unlikely to understand jargon or specialist notations. In fact, a form that users can interact with is most effective, and building prototypes is an extremely powerful approach (see Box 9.2 and [Chapter 11](#) for more on prototyping).

In the rest of this section, we explore in more depth the significance and practicality of involving users in design, i.e. using a user-centered approach, and consider again the four activities of interaction design that were introduced in [Chapter 1](#).

BOX 9.2

The Value of Prototyping

I learned the value of prototyping through a very effective role-playing exercise. I was on a course designed to introduce new graduates to different possible careers in industry. One of the themes was production and manufacturing and the aim of one group exercise was to produce a book. Each group was told that it had 30 minutes to deliver 10 books to the person in charge. Groups were given various pieces of paper, scissors, sticky tape, staples, etc., and told to organize ourselves as best we could. So in my group we set to work organizing ourselves into a production line, with one of us cutting up the paper, another stapling the pages together, another sealing the binding with the sticky tape, and so on. One person was even in charge of quality assurance. It took us less than 10 minutes to produce the 10 books, and we rushed off with our delivery. When we showed the person in charge, he replied, 'That's not what I wanted; I need it bigger than that.' Of course, the size of the notebook wasn't specified in the description of the task, so we found out how big he wanted it, got some more materials, and scooted back to produce 10 more books. Again, we set up our production line and produced 10 books to the correct size. On delivery, we were again told that it was not what was required: he wanted the binding down the other edge. This time we got as many of the requirements as we could and went back, developed one book, and took that back for further feedback and refinement before producing the 10 required.

If we had used prototyping as a way of exploring our ideas and checking requirements in the first place, we could have saved so much effort and resource! ■

9.2.1 The Importance of Involving Users

The description above emphasizes the need to involve users in interaction design, but why is it important? Before the impact that user involvement can have on project success was recognized, it was common for developers to talk to managers or to proxy users, i.e. people who role-played as users, when eliciting requirements, or even to use their own judgment without reference to anyone else. While a proxy user can provide useful information, they will not have the same perspective as someone who performs the task every day, or who will use the intended product on a regular basis. For example, several years ago, I was involved with a system to process and record financial transactions from the foreign exchange (forex) dealers in a large international bank. The users of this system took the handwritten transaction records completed by the forex dealers and entered the details into the system. The system then validated the transaction and communicated a confirmation to the relevant parties. When the requirements for this system were developed, no one from the development team spoke to the end-users of the system; the requirements were identified by higher level managers. Although the system was successfully used to support the task, the end-users had developed several work-arounds and crib sheets. For example, each desk had a sheet of paper with lists of buyer codes and names. The system required both buyer code and buyer name to be entered, but it was quite common for the dealers to write only the buyer name on the transaction record. The list of names and codes was used to identify the codes manually.

The best way to ensure that development continues to take users' activities into account is to involve real users throughout development. In this way, developers can gain a better understanding of users' goals, leading to a more appropriate, more usable product. However, two other aspects that have nothing to do with functionality are equally as important if the product is to be usable and used: expectation management and ownership.

Expectation management is the process of making sure that the users' expectations of the new product are realistic. The purpose of expectation management is to ensure that there are no surprises for

users when the product arrives. If users feel they have been cheated by promises that have not been fulfilled, then this will cause resistance and even rejection. Marketing of the new arrival must be careful not to misrepresent the product, although it may be particularly difficult to achieve with a large and complex system (Nevo and Wade, 2007). How many times have you seen an advert for something you thought would be really good to have, but when you see one, discover that the marketing hype was a little exaggerated? I expect you felt quite disappointed and let down. This is the kind of feeling that expectation management tries to avoid.

It is better to exceed users' expectations than to fall below them. This does not mean adding more features, but that the product supports the users' goals more effectively than they expect. Involving users throughout development helps with expectation management because they can see from an early stage what the product's capabilities are. They will also understand better how it will affect their jobs and lives, and why the features are designed that way. Adequate and timely training is another technique for managing expectations. If users have the chance to work with the product before it is released, through training or hands-on demonstrations of a pre-release version, then they will understand better what to expect when the final product is available.

A second reason for user involvement is ownership. Users who are involved and feel that they have contributed to a product's development are more likely to feel a sense of ownership towards it and support its use.

9.2.2 Degrees of User Involvement

Different degrees of user involvement may be implemented in order to manage expectations and to create a feeling of ownership. At one end of the spectrum, users may be co-opted to the design team so that they are major contributors. For any one user, this may be on a full-time basis or a part-time basis, and it may be for the duration of the project or for a limited time only. There are advantages and disadvantages to each situation. If a user is co-opted full-time for the whole project, their input will be consistent and they will become very

familiar with the product and its rationale. However, if the project takes many years, they may lose touch with the rest of the user group, making their input less valuable. If a user is co-opted part-time for the whole project, she will offer consistent input to development while remaining in touch with other users. Depending on the situation, this will need careful management as the user will be trying to learn new jargon and handle unfamiliar material as a member of the design team, yet concurrently trying to fulfill the demands of her original job. This can become very stressful for the individuals. If a number of users from each user group are co-opted part-time for a limited period, input is not necessarily consistent across the whole project, but careful coordination between users can alleviate this problem. In this case, one user may be part of the design team for 6 months, then another takes over for the next 6 months, and so on.

At the other end of the spectrum, users may simply be kept informed through regular newsletters or other channels of communication. Provided they are able to influence the development process through workshops or similar events, this can be an effective approach to expectation management and ownership. In a situation with hundreds or even thousands of users, it would not be feasible to involve them all as members of the team, and so this might be the only viable option. In this case a compromise situation is probably the best. Representatives from each user group may be co-opted onto the team on a full-time basis, while other users are involved through design workshops, evaluation sessions, and other data-gathering activities.

The individual circumstances of the project affect what is realistic and appropriate. If your end-user groups are identifiable, e.g. you are developing a product for a particular company, then it is easier to involve them. If, however, you are developing a product for the open market, it is unlikely that you will be able to co-opt a user to your design team, and so alternative approaches are needed. Box 9.3 outlines an alternative way to obtain user input.

How actively users should be involved is a matter for debate. Some studies have shown that too much user involvement can lead to

problems. This issue is discussed in the next Dilemma box.

BOX 9.3

Ongoing User Involvement after a Product is Released

Once a product has been released and the focus of development moves to future versions, a different kind of user involvement is possible – one that captures data about the real use of the product. This may be obtained in a number of ways, e.g. through interaction between users and customer service agents, or through automated error reporting systems. For example, Microsoft has millions of customers around the world, about 30% of whom call their customer support lines with problems and frustrations resulting from poor features or software errors. This data about customer behavior and their problems with the products is fed back into product development and improvement (Cusumano and Selby, 1995).

Error reporting systems (ERS, also called online crashing analysis) automatically collect information from users (with their permission), which is used to improve applications in the longer term. [Figure 9.2](#) shows two typical dialog boxes for the Windows error reporting system that is built into Microsoft operating systems (Glerum *et al.*, 2009). ■

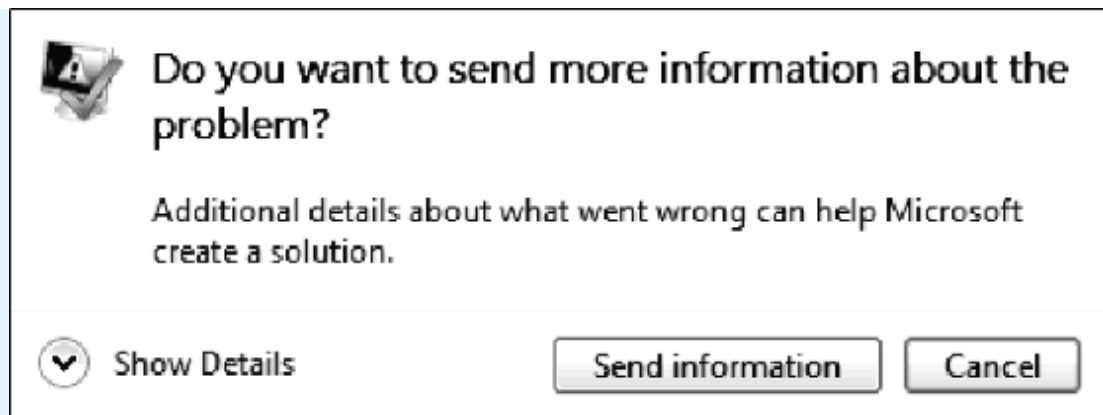


Figure 9.2 Two typical dialog boxes from the Windows error reporting system. This kind of reporting can have a significant effect on the quality of applications: for example, 29% of the errors fixed by the Windows XP (Service Pack 1) team were based on information collected through their ERS

Dilemma

Too Much of a Good Thing?

Involving users in development is a good thing. Or is it? And how

much should they become involved? In what role(s)? Are users qualified to lead a technical development project? Or is it more appropriate for their role to be confined to evaluating prototypes? Involving users to any degree incurs costs, so what evidence is there that user involvement is productive, or that it is worth putting the required level of resources into it?

Research by Keil and Carmel (1995) indicates that the more successful projects have direct links to users and customers, while Kujala and Mäntylä (2000) concluded that user studies conducted early in development produce benefits that outweigh the costs of conducting them. Subrayaman *et al* (2010) found a mixed result. They investigated the impact of user participation on the satisfaction with the product by both developers and users. They found that for new products, developer satisfaction increased as user participation increased, while user satisfaction was higher where participation was low, but satisfaction dropped as participation increased. For maintenance projects, both developers and users were most satisfied with a moderate level of participation (approximately 20% of overall project development time). If we focus just on the user satisfaction as an indication of project success, then it seems that low user participation is most beneficial.

According to some research, a low level of user involvement is beneficial whereas too much can lead to problems. For example, Heinbokel *et al* (1996) found that high user involvement projects tended to run less smoothly. Subrayaman *et al* (2010) also identified that high levels of user involvement can generate unnecessary conflicts and increased reworking.

The kind of product being developed, the kind of user involvement possible, and the application domain all influence the impact that users can have on a project. Scaife *et al* (1997) suggest that involving different kinds of user at different stages of design yields positive results, while Wagner and Piccoli (2007) provide evidence that user involvement is most effective when the product becomes salient to their daily lives, which is often close to or even after deployment.

Abelein *et al* (2013) performed a detailed review of the literature in this area and concluded that, overall, the evidence indicates user involvement has a positive effect on user satisfaction and system use. However, they also found that even though the data clearly indicates this positive effect, some links have a large variation, suggesting that there is still no clear conceptual model to measure the effects consistently. In addition, they found that most studies with negative correlations involving users and system success were published over a decade ago.

While user involvement is widely acclaimed as beneficial, there are still many dimensions that need to be considered and balanced. ■

9.2.3 What Is a User-Centered Approach?

Throughout this book, we emphasize the need for a user-centered approach to development. By this we mean that the real users and their goals, not just technology, are the driving force behind product development. As a consequence, a well-designed system will make the most of human skill and judgment, will be directly relevant to the activity in hand, and will support rather than constrain the user. This is less of a technique and more of a philosophy.

In 1985, Gould and Lewis (1985) laid down three principles they believed would lead to a ‘useful and easy to use computer system’:

1. *Early focus on users and tasks.* This means first understanding who the users will be by directly studying their cognitive, behavioral, anthropomorphic, and attitudinal characteristics. This requires observing users doing their normal tasks, studying the nature of those tasks, and then involving users in the design process.
2. *Empirical measurement.* Early in development, the reactions and performance of intended users to printed scenarios, manuals, etc., are observed and measured. Later on, users interact with simulations and prototypes and their performance and reactions are observed, recorded, and analyzed.

3. *Iterative design*. When problems are found in user testing, they are fixed and then more tests and observations are carried out to see the effects of the fixes. This means that design and development is iterative, with cycles of design–test–measure–redesign being repeated as often as necessary.

These three principles are now accepted as the basis for a user-centered approach (e.g. see Mao *et al*, 2005) but when Gould and Lewis wrote their paper, they were not accepted by most developers. In fact, they remark in their paper that when they started recommending these to designers, the designers' reactions implied that these principles were obvious. However, when they asked designers at a human factors symposium for the major steps in software design, most of them did not cite many of the principles – in fact, only 2% mentioned all of them. So maybe they had obvious merit, but they were not so easy to put into practice. The Olympic Messaging System (OMS) (Gould *et al*, 1987) was the first reported large computer-based system to be developed using these three principles. Here a combination of techniques was used to elicit users' reactions to designs, from the earliest prototypes through to the final product. In this case, users were mainly involved in evaluating designs. Below, we discuss these principles in more detail.

Early Focus on Users and Tasks

This principle can be expanded and clarified through the following five further principles:

1. Users' tasks and goals are the driving force behind the development.

In a user-centered approach to design, while technology will inform design options and choices, it should not be the driving force. Instead of saying 'Where can we deploy this new technology?' say 'What technologies are available to provide better support for users' goals?'

2. Users' behavior and context of use are studied and the system is designed to support them.

This is about more than just capturing the tasks and the users' goals. How people perform their tasks is also significant. Understanding behavior highlights priorities, preferences, and implicit intentions. One argument against studying current behavior is that we are looking to improve work, not to capture bad habits in automation. The implication is that exposing designers to users is likely to stifle innovation and creativity, but experience tells us that the opposite is true (Beyer and Holtzblatt, 1998). In addition, if something is designed to support an activity with little understanding of the real work involved, it is likely to be incompatible with current practice, and users don't like to deviate from their learned habits if operating a new device with similar properties (Norman, 2013).

3. Users' characteristics are captured and designed for.

When things go wrong with technology, we often say that it is our fault. But as humans, we are prone to making errors and we have certain limitations, both cognitive and physical. Products designed to support humans should take these limitations into account and should limit the mistakes we make. Cognitive aspects such as attention, memory, and perception issues were introduced in [Chapter 3](#). Physical aspects include height, mobility, and strength. Some characteristics are general, such as about one man in 12 has some form of color blindness, but some characteristics may be associated more with the job or particular task at hand. So, as well as general characteristics, we need to capture those specific to the intended user group.

4. Users are consulted throughout development from earliest phases to the latest and their input is seriously taken into account. As discussed above, there are different levels of user involvement and there are different ways in which to consult users. However involvement is organized, it is important that users are respected by designers.

5. All design decisions are taken within the context of the users, their work, and their environment. This does not necessarily mean that users are actively involved in design decisions.

As long as designers remain aware of the users while making their decisions, then this principle will be upheld. Keeping this context in mind can be difficult, but using personas is one way to achieve this (see [Chapter 10](#)); an easily accessible collection of gathered data is another. Some design teams set up a specific design room for the project where data and informal records of brainstorming sessions are pinned on the walls or left on the table.

Activity 9.2

Assume that you are involved in developing a novel online experience for buying garden plants. Although several websites exist for buying plants online, you want to produce a distinct experience to increase the organization's market share. Suggest ways of applying the above principles in this task.

Comment

Show/Hide

Empirical Measurement

Specific usability and user experience goals should be identified, clearly documented, and agreed upon at the beginning of the project. They can help designers to choose between alternative designs and to check on progress as the product is developed. Identifying specific goals up front means that the product can be empirically evaluated at regular stages as it is developed.

Iterative Design

Iteration allows designs to be refined based on feedback. As users and designers engage with the domain and start to discuss requirements, needs, hopes, and aspirations, then different insights into what is needed, what will help, and what is feasible will emerge. This leads to a need for iteration, for the activities to inform each

other and to be repeated. However good the designers are and however clear the users may think their vision is of the required artifact, it will be necessary to revise ideas in light of feedback, several times. This is particularly true when trying to innovate. Innovation rarely emerges whole and ready to go. It takes time, evolution, trial and error, and a great deal of patience. Iteration is inevitable because designers never get the solution right the first time (Gould and Lewis, 1985).

9.2.4 Four Basic Activities of Interaction Design

Four basic activities for interaction design were introduced in [Chapter 1](#), some of which you will have engaged in when doing Activity 9.1. These are: establishing requirements for the user experience, designing alternatives that meet those requirements, prototyping the alternative designs so that they can be communicated and assessed, and evaluating what is being built throughout the process and the user experience it offers. They are fairly generic activities and can be found in other design disciplines too, such as graphic design, architectural design, and product design.

Establishing Requirements

In order to design something to support people, we must know who our target users are and what kind of support an interactive product could usefully provide. These needs form the basis of the product's requirements and underpin subsequent design and development. This activity is fundamental to a user-centered approach, and is very important in interaction design. Understanding these needs is gleaned through data gathering and analysis, which are discussed in [Chapters 7](#) and [8](#). The requirements activity is discussed further in [Chapter 10](#).

Designing Alternatives

This is the core activity of designing: actually suggesting ideas for meeting the requirements. This activity can be viewed as two sub-activities: conceptual design and concrete design. Conceptual

design involves producing the conceptual model for the product, and a conceptual model describes an abstraction outlining what people can do with a product and what concepts are needed to understand how to interact with it. Concrete design considers the detail of the product including the colors, sounds, and images to use, menu design, and icon design. Alternatives are considered at every point. You met some of the ideas for conceptual design in [Chapter 2](#), and some more design issues for specific interface types in [Chapter 6](#); we go into more detail about how to design an interactive product in [Chapter 11](#).

Prototyping

Interaction design involves designing interactive products. The most sensible way for users to evaluate such designs is to interact with them, and this can be achieved through prototyping. This does not necessarily mean a piece of software is required. There are different prototyping techniques, not all of which require a working piece of software. For example, paper-based prototypes are very quick and cheap to build and are very effective for identifying problems in the early stages of design, and through role-playing users can get a real sense of what it will be like to interact with the product. Prototyping is also covered in [Chapter 11](#).

Evaluating

Evaluation is the process of determining the usability and acceptability of the product or design that is measured in terms of a variety of usability and user experience criteria. Interaction design requires a high level of user involvement throughout development, and this enhances the chances of an acceptable product being delivered. Evaluation does not replace the activities concerned with quality assurance and testing to make sure that the final product is fit for purpose, but it complements and enhances them. We devote [Chapters 13](#) to [15](#) to evaluation.

The activities of establishing requirements, designing alternatives, building prototypes, and evaluating them are intertwined: alternatives are evaluated through the prototypes and the results are fed back

into further design or might identify missing requirements. This iteration is one of the key characteristics of a user-centered approach.

9.2.5 A Simple Lifecycle Model for Interaction Design

Understanding what activities are involved in interaction design is the first step to being able to do it, but it is also important to consider how the activities are related to one another so that the full development process can be seen. The term lifecycle model is used to represent a model that captures a set of activities and how they are related. Software engineering has spawned many lifecycle models including the waterfall, spiral, and RAD (rapid applications development) models (Sommerville, 2010). One lifecycle model that was not widely used, but which was intended to capture the iterative nature of software development, was called the fountain lifecycle model (Henderson-Sellers and Edwards, 1993). The field of HCI has also been associated with several lifecycle models such as the Star (Hartson and Hix, 1989) and an international standard model ISO 9241-210.

Existing models have varying levels of sophistication and complexity. For projects involving only a few experienced developers, a simple process would probably be adequate. However, for larger systems involving tens or hundreds of developers with hundreds or thousands of users, a simple process just isn't enough to provide the management structure and discipline necessary to engineer a usable product. So something is needed that will provide more formality and more discipline (described in Box 9.1 as the systems design approach to interaction design). Note that this does not necessarily mean that innovation is lost or that creativity is stifled, just that a structured process is used to provide a more stable framework for creativity.



However simple or complex it appears, any lifecycle model is a simplified version of reality. It is intended as an abstraction and, as with any good abstraction, only the amount of detail required for the task at hand will be included. Any organization wishing to put a lifecycle model into practice will need to add detail specific to its particular circumstances and culture.

The activities of interaction design are related as shown in [Figure 9.3](#). This model incorporates the four activities of interaction design and the three principles of user-centered design discussed above. Depending on the kind of product being developed, it may not be possible or appropriate to follow this model for every aspect, and more detail would be required to put the lifecycle into practice in a real project. We have not specified outputs from each activity, although measurable usability and user experience criteria would be specified early on and referred to across all the activities.

The model is not intended to be prescriptive; that is, we are not suggesting that this is how all interactive products are or should be developed. It is based on our observations of interaction design and on information we have gleaned in the research for this book. It has its roots in the software engineering and HCI lifecycle models

mentioned above and it represents what we believe is practiced in the field.

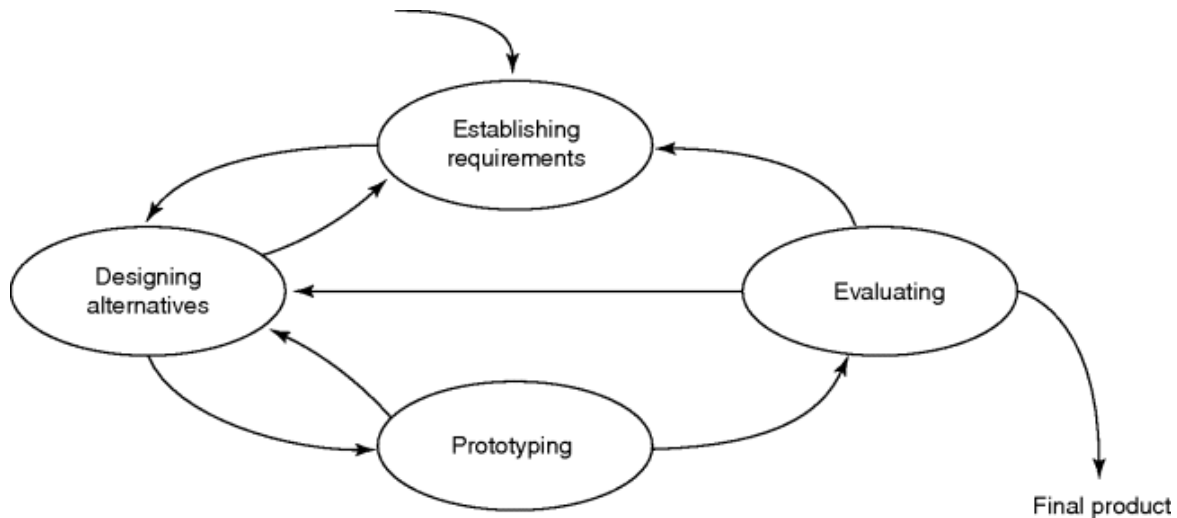


Figure 9.3 A simple interaction design lifecycle model

Most projects start by establishing requirements. The project may have arisen because of some evaluation that has been done, but the lifecycle of the new (or modified) product can be thought of as starting at this point. From this activity, some alternative designs are generated in an attempt to meet the requirements that have been identified. Then prototype versions of the designs are developed and evaluated. Based on the feedback from the evaluations, the team may need to return to identify more needs or refine requirements, or it may go straight into redesigning. It may be that more than one alternative design follows this iterative cycle in parallel with others, or it may be that one alternative at a time is considered. Implicit in this cycle is that the final product will emerge in an evolutionary fashion from a rough initial idea through to the finished product. Exactly how this evolution happens may vary from project to project, and we return to this issue in [Chapter 11](#). The only factor limiting the number of times through the cycle is the resources available, but whatever the number is, development ends with an evaluation activity that ensures the final product meets the prescribed user experience and usability criteria.

9.3 Some Practical Issues

The discussion so far has highlighted some issues about the practical application of user-centered design and the simple lifecycle of interaction design that we have introduced. These issues must be addressed in order to be able to do interaction design in practice. We capture these issues in the following questions:

- Who are the users?
- What do we mean by needs?
- How do you generate alternative designs?
- How do you choose among alternatives?
- How do you integrate interaction design activities with other lifecycle models?

9.3.1 Who Are the Users?

With all this emphasis on users and user involvement in the interaction design process, a fairly basic question to ask is ‘Who are the users?’.

Identifying the users may seem like a straightforward activity, but in fact there are many interpretations of the term ‘user,’ and involving the right users is crucial to successful user-centered design. The most obvious definition is those people who interact directly with the product to achieve a task. Most people would agree with this definition; however, there are others who can also be thought of as users. For example, Holtzblatt and Jones (1993) include in their definition of users those who manage direct users, those who receive products from the system, those who test the system, those who make the purchasing decision, and those who use competitive products. Eason (1987) identifies three categories of user: primary, secondary, and tertiary. Primary users are those likely to be frequent hands-on users of the system; secondary users are occasional users or those who use the system through an intermediary; and tertiary

users are those affected by the introduction of the system or who will influence its purchase.

The trouble is that there is a surprisingly wide collection of people who all have a stake in the development of a successful product. These people are called stakeholders. Stakeholders are “people or organizations who will be affected by the system and who have a direct or indirect influence on the system requirements” (Kotonya and Sommerville, 1998). Dix *et al* (2004) make an observation that is very pertinent to a user-centered view of development: “It will frequently be the case that the formal ‘client’ who orders the system falls very low on the list of those affected. Be very wary of changes which take power, influence or control from some stakeholders without returning something tangible in its place.”

The group of stakeholders for a particular product will be larger than the group of people normally thought of as users, although it will of course include users. Using the definition above, the group of stakeholders includes the development team itself as well as its managers, the direct users and their managers, recipients of the product's output, people who may lose their jobs because of the introduction of the new product, and so on (Sharp *et al*, 1999).

For example, consider again the travel planner in Activity 9.1. According to the description we gave you, the user group for the system has just one member: you. However, the stakeholders for the system would also include the people you are going to visit, the airlines you book flights with, staff in the hotels you might stay at, a wide selection of companies and staff members who have an interest to make sure that any information you are given is correct, and even the restaurants on the route chosen for your journey, since the route suggested by the system will determine whether or not you drive past those restaurants.

Identifying the stakeholders for your project means that you can make an informed decision about who should be involved and to what degree, but how to make sure that you include stakeholders who are relevant is more complex. Alexander and Robertson (2004) suggest using an onion diagram to model stakeholders and their

involvement. This diagram shows concentric circles of stakeholder zones with the product being developed sitting in the middle. Lim *et al* (2010) developed a process and supporting tool called StakeNet which relies on recommendations through social networking to identify and prioritize relevant stakeholders. Ellen Gottesdiener expands on the role of stakeholders in her interview at the end of this chapter.

Activity 9.3

Who are the stakeholders for an automated check-out system of a large supermarket?

Comment

Show/Hide

9.3.2 What Do We Mean by 'Needs'?

If you had asked someone in the street in the late 1990s what she needed, I doubt that the answer would have included interactive television, or a ski jacket with integrated MP3 player, or a robot pet. If you presented the same person with these possibilities and asked whether she would buy them if they were available, then the answer may have been more positive. When we talk about identifying needs, it is not simply a question of asking people, 'What do you need?' and then supplying it, because people don't necessarily know what is possible. Robertson and Robertson (2013) refer to 'un-dreamed-of' needs, which are those that users are unaware they could have. Instead, we have to approach it by understanding the characteristics and capabilities of the users, what they are trying to achieve, how they achieve it currently, and whether they would achieve their goals more effectively and have a more enjoyable experience if they were supported differently.

There are many dimensions along which a user's characteristics and capabilities may vary, and that will have an impact on the product's

design. You have met some of the cognitive ones in [Chapter 3](#). A person's physical characteristics may also affect the design: size of hands may affect the size and positioning of input buttons, and motor abilities may affect the suitability of certain input and output devices; height is relevant in designing a physical kiosk, for example; and strength in designing a child's toy – a toy should not require too much strength to operate, but may require strength greater than expected for the target age group to change batteries or perform other operations suitable only for an adult. Cultural diversity and experience may affect the terminology the intended user group is used to, or how nervous about technology a set of users may be, or how a facility is used (we discuss user requirements in more detail in [Chapter 10](#)).

If a product is a new invention, then it can be difficult to identify the users and representative tasks for them, e.g. before in-car navigation systems were first developed, there were no users to consult about requirements and there were no representative tasks to identify. Those developing the system had to imagine who might want to use it and what they might want to do with it.

It may be tempting for designers simply to design what they would like to use themselves, but their ideas would not necessarily coincide with those of the target user group, because they have different experiences and expectations. It is imperative that representative users from the real target group be consulted. For example, Netpliance developed a new product that would seamlessly integrate all the services necessary for the user to achieve a specific task on the Internet (Isensee *et al*, 2000). They took a user-centered approach and employed focus group studies and surveys to understand their customers' needs, but developers observed the focus groups to learn more about their intended user group. Isensee *et al* (p. 60) comment that, "It is always tempting for developers to create products they would want to use or similar to what they have done before."

Whether the product is a new invention or not, it is always useful to start by understanding similar behavior that is already established. Introducing something new into people's lives, especially a new

everyday item, requires a culture change in the target user population, and it takes a long time to effect a culture change.

Focusing on people's goals and on usability and user experience goals is a more promising approach to interaction design than focusing on people's needs and expecting them to be able to tell us the requirements for a product. Techniques for data gathering to investigate these goals and to establish requirements are discussed more in [Chapters 7](#) and [10](#).

9.3.3 How Do You Generate Alternative Designs?

A common human tendency is to stick with something that we know works. We recognize that a better solution may exist out there somewhere, but it is very easy to accept this one because we know it works – it is ‘good enough.’ Settling for a solution that is good enough is not necessarily bad, but it may be undesirable because good alternatives may never be considered, and considering alternative solutions is a crucial step in the process of design. But where do these alternative ideas come from?

One answer to this question is that they come from the individual designer's flair and creativity (the genius design described in Box 9.1). Although it is certainly true that some people are able to produce wonderfully inspired designs while others struggle to come up with any ideas at all, very little in this world is completely new. For example, if you think of something commonly believed to be an invention, such as the steam engine, this was in fact inspired by the observation that the steam from a kettle boiling on the stove lifted the lid. Clearly there was an amount of creativity and engineering involved in making the jump from a boiling kettle to a steam engine, but the kettle provided the inspiration to translate experience gained in one context into a set of principles that could be applied in another. Innovations usually arise through cross-fertilization of ideas from different perspectives, individuals, and contexts; the evolution of an existing product through use and observation; or straightforward copying of other, similar products.

Cross-fertilization may result by discussing ideas with other designers, while Buxton (2007) reports that different perspectives from users generated original ideas about alternative designs. As an example of evolution, consider the word processor. The capabilities of suites of office software have gradually increased from the time they first appeared. Initially, a word processor was just an electronic version of a typewriter, but gradually other capabilities, including the spell-checker, thesaurus, style sheets, graphical capabilities, and so on, were added.

Although creativity and invention are often wrapped in mystique, we do understand something of the process and of how creativity can be enhanced or inspired. We know, for instance, that browsing a collection of designs will inspire designers to consider alternative perspectives, and hence alternative solutions. The field of case-based reasoning (Maher and Pu, 1997) emerged from the observation that designers solve new problems by drawing on knowledge gained from solving previous similar problems. As Schank (1982, p. 22) puts it, “An expert is someone who gets reminded of just the right prior experience to help him in processing his current experiences.” And while those experiences may be the designer's own, they can equally well be others’.

Another approach to creativity has been taken by Maiden *et al* (2007a). They have been running creativity workshops to generate innovative requirements in an air traffic management (ATM) application domain. Their idea is to introduce experts in different fields into the workshop, and then invite stakeholders to identify analogies between their own field and this new one. For example, they have invited an Indian textile expert, a musician, a TV program scheduler, and a museum exhibit designer. Although not all obviously analogical domains, they sparked creative ideas for the air traffic management application. For example, participants reported that one textile design was elegant, i.e. simple, beautiful, and symmetrical. They then transferred these properties to a key area of the ATM domain – that of aircraft conflict resolution. They explored the meaning of elegance within this context, and realized that elegance is perceived differently by different controllers. From this

they generated the requirement that the system should be able to accommodate different air traffic controller styles during conflict resolution.

A more pragmatic answer to this question, then, is that alternatives come from seeking different perspectives and looking at other designs. The process of inspiration and creativity can be enhanced by prompting a designer's own experience and studying others' ideas and suggestions. Deliberately seeking out suitable sources of inspiration is a valuable step in any design process. These sources may be very close to the intended new product, such as competitors' products, or they may be earlier versions of similar systems, or something completely different.

Having said this, under some circumstances the scope to consider alternative designs may be limited. Design is a process of balancing constraints and constantly trading off one set of requirements with another, and the constraints may be such that there are very few viable alternatives available. For example, if you are designing a software system to run under the Windows operating system, then elements of the design will be prescribed because you must conform to the Windows look and feel, and to other constraints intended to make Windows programs consistent for the user. If you are producing an upgrade to an existing system, then you may want to keep the familiar elements of it and retain basically the same user experience.

Activity 9.4

Consider again the travel planner introduced at the beginning of the chapter. Reflecting on the process again, what do you think inspired your outline design? See if you can identify any elements within it that you believe are truly innovative.

Comment

Show/Hide

BOX 9.4

A Box Full of Ideas

The innovative product design company IDEO was introduced in [Chapter 1](#). Underlying some of their creative flair is a collection of weird and wonderful engineering housed in a large flatbed filing cabinet called the TechBox (see [Figure 9.4](#)). The TechBox holds hundreds of gizmos and interesting materials, divided into categories such as: Amazing Materials, Cool Mechanisms, Interesting Manufacturing Processes, Electronic Technologies, and Thermal and Optical. Each item has been placed in the box because it represents a neat idea or a new process. Staff at IDEO take along a selection of items from the TechBox to brainstorming meetings. The items may be chosen because they provide useful visual props or possible solutions to a particular issue, or simply to provide some light relief.

Each item is clearly labeled with its name and category, but further information can be found by accessing the TechBox's online catalog. Each item has its own page detailing what the item is, why it is interesting, where it came from, and who has used it or knows more about it. Items in the box include an example of metal-coated wood, and materials with and without holes that stretch, bend, and change shape or color at different temperatures.

Each of IDEO's offices has a TechBox and each TechBox has its own curator who is responsible for maintaining and cataloging the items and for promoting its use within the office. Anyone can submit a new item for consideration, and as items become commonplace they are removed from the TechBox to make way for the next generation of fascinating curios. ■



Figure 9.4 The TechBox at IDEO

Source: Reproduced by permission of IDEO. Photo by Jorge Davies.

Dilemma

Copying for Inspiration: Is it Legal?

Designers draw on their experience of design when approaching a new project. This includes the use of previous designs that they know work, both designs they have created themselves and those that others have created. Others' creations often spark inspiration that also leads to new ideas and innovation. This is well known and understood. However, the expression of an idea

is protected by copyright, and people who infringe that copyright can be taken to court and prosecuted. Note that copyright covers the expression of an idea and not the idea itself. This means, for example, that while there are numerous MP3 players all with similar functionality, this does not represent an infringement of copyright as the idea has been expressed in different ways, and it is the expression that has been copyrighted. Copyright is free and is automatically invested in the author of something, e.g. the writer of a book or a programmer who develops a program, unless he signs the copyright over to someone else. People who produce something through their employment, such as programs or products, may have in their employment contract a statement saying that the copyright relating to anything produced in the course of that employment is automatically assigned to the employer and does not remain with the employee.

Patenting is an alternative to copyright that does protect the idea rather than the expression. There are various forms of patenting, each of which is designed to allow the inventor the chance to capitalize on an idea. It is unusual for software to be patented, since it is a long, slow, and expensive process, although there have been some examples of patenting business processes. For example, Amazon has patented its one-click purchasing process, which allows regular users simply to choose a purchase and buy it with one mouse click (US Patent No. 5960411, September 29, 1999). This is possible because the system stores its customers' details and recognizes them when they access the Amazon site again.

In recent years, the creative commons community (creativecommons.org) has suggested more flexible licensing arrangements that allow others to reuse and extend a piece of created work, thereby supporting collaboration. In the Open Source software development movement, for example, software code is freely distributed and can be modified, incorporated into other software, and redistributed under the same open source conditions. No royalty fees are payable on any use of open source code. These movements do not replace copyright or

patent law, but they provide an alternative route for the dissemination of ideas.

So the dilemma comes in knowing when it is OK to use someone else's work as a source of inspiration and when you are infringing copyright or patent law. The issues around this question are complex and detailed, and well beyond the scope of this book, but up-to-date information and examples of law cases that have been brought successfully and unsuccessfully can be found in Bainbridge (2014). ■

9.3.4 How Do You Choose Among Alternative Designs?

Choosing among alternatives is about making design decisions: Will the device use keyboard entry or a touch screen? Will the product provide an automatic memory function or not? These decisions will be informed by the information gathered about users and their tasks, and by the technical feasibility of an idea. Broadly speaking, though, the decisions fall into two categories: those that are about externally visible and measurable features, and those that are about characteristics internal to the system that cannot be observed or measured without dissecting it. For example, in a photocopier, externally visible and measurable factors include the physical size of the machine, the speed and quality of copying, the different sizes of paper it can use, and so on. Underlying each of these factors are other considerations that cannot be observed or studied without dissecting the machine. For example, the choice of materials used in a photocopier may depend on its friction rating and how much it deforms under certain conditions.

In an interactive product there are similar factors that are externally visible and measurable and those that are hidden from the users' view. For example, exactly why it takes 30 seconds for a web page to load, or why it takes an hour for a cell phone text message to arrive, will be influenced by technical decisions made when the web page or cell phone software was constructed. From the users' viewpoint the important observation is the fact that it does take 30 seconds to load or an hour to arrive.

In interaction design, the way in which the users interact with the product is considered the driving force behind the design and so we concentrate on the externally visible and measurable behavior. Detailed internal workings are still important to the extent that they affect external behavior or features.

One answer to the question posed above is that we choose between alternative designs by letting users and stakeholders interact with them and by discussing their experiences, preferences, and suggestions for improvement. This is fundamental to a user-centered approach to development. This in turn means that the designs must be available in a form that can be reasonably evaluated with users, not in technical jargon or notation that seems impenetrable to them.

One form traditionally used for communicating a design is documentation, e.g. a description of how something will work or a diagram showing its components. The trouble is that a static description cannot easily capture the dynamics of behavior, and for an interactive product we need to communicate to the users what it will be like to actually operate it.

In many design disciplines, prototyping is used to overcome potential client misunderstandings and to test the technical feasibility of a suggested design and its production. Prototyping involves producing a limited version of the product with the purpose of answering specific questions about the design's feasibility or appropriateness. Prototypes give a better impression of the user experience than simple descriptions, and there are different kinds of prototyping that are suitable for different stages of development and for eliciting different kinds of information. Prototyping is discussed in detail in [Chapter 11](#).

Another basis on which to choose between alternatives is quality, but this requires a clear understanding of what quality means. People's views of what is a quality product vary. Whenever we use anything we have some notion of the level of quality we are expecting, wanting, or needing. Whether this level of quality is expressed formally or informally does not matter. The point is that it exists and we use it consciously or subconsciously to evaluate alternative

items. For example, if you have to wait too long to download a web page, then you are likely to give up and try a different site – you are applying a certain measure of quality associated with the time taken to download the web page. If one smartphone makes it easy to access your favorite music channel while another involves several complicated key sequences, then you are likely to buy the former rather than the latter. Here, you are applying a quality criterion concerned with efficiency.

If you are the only user of a product, then you don't necessarily have to express your definition of quality since you don't have to communicate it to anyone else. However, as we have seen, most projects involve many different stakeholder groups, and you will find that each of them has a different definition of quality and different acceptable limits for it. For example, although all stakeholders may agree on targets such as 'response time will be fast' or 'the menu structure will be easy to use,' exactly what each of them means by this is likely to vary. Disputes are inevitable when, later in development, it transpires that 'fast' to one set of stakeholders meant 'under a second' while to another it meant 'between 2 and 3 seconds.' Capturing these different views in clear unambiguous language early in development takes you halfway to producing a product that will be well-regarded by all your stakeholders. It helps to clarify expectations, provides a benchmark against which products of the development process can be measured, and gives you a basis on which to choose among alternatives.

The process of writing down formal, verifiable – and hence measurable – usability criteria is a key characteristic of an approach to interaction design called usability engineering. This has emerged over many years and with various proponents (Whiteside *et al*, 1988; Nielsen, 1993). Usability engineering involves specifying quantifiable measures of product performance, documenting them in a usability specification, and assessing the product against them. One way in which this approach is used is to make changes to subsequent versions of a system based on feedback from carefully documented results of usability tests for the earlier version.

Activity 9.5

Consider the travel planner that you designed in Activity 9.1. Suggest some usability criteria that you could use to determine the planner's quality. Use the usability goals introduced in [Chapter 1](#): effectiveness, efficiency, safety, utility, learnability, and memorability. Be as specific as possible. Check your criteria by considering exactly what you would measure and how you would measure its performance.

Then try to do the same thing for some of the user experience goals introduced in [Chapter 1](#) (these relate to whether a system is satisfying, enjoyable, motivating, rewarding, and so on).

Comment

Show/Hide

9.3.5 How Do You Integrate Interaction Design Activities with Other Lifecycle Models?

There are several lifecycle models associated with other disciplines that contribute to interaction design (see [Figure 1.4](#)). Prominent among these lifecycle models are those associated with software engineering. Discussion about how best to integrate user-centered design and software engineering, and how to raise awareness of user-centered techniques with software engineers has been ongoing for several years, e.g. see Seffah *et al* (2005).

The latest, and some would argue the most promising, attempts at integration focus on a relatively recent trend in software engineering, called agile software development. Agile methods began to emerge in the late 1990s. The most well known of these are eXtreme Programming (Beck and Andres, 2005), Crystal (Cockburn, 2005), Scrum (Schwaber and Beedle, 2002), and Adaptive Software Development (ASD) (Highsmith, 2000). Dynamic Systems Development Method (DSDM) (DSDM, 2014), although established

before the current agile movement, also belongs to the agile family as it adheres to the agile manifesto (reproduced below). These methods differ, but they all stress the importance of iteration, early and repeated user feedback, being able to handle emergent requirements, and striking a good balance between flexibility and structure. They also all emphasize collaboration, face-to-face communication, streamlined processes to avoid unnecessary activities, and the importance of practice over process, i.e. of getting work done.

The opening statement for the *Manifesto for Agile Software Development* (www.agilemanifesto.org/) is:

We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value:

Individuals and interactions over processes and tools

Working software over comprehensive documentation

Customer collaboration over contract negotiation

Responding to change over following a plan

This manifesto is underpinned by a series of principles, which range from communication with the business through to excellence of coding and maximizing the amount of work done. The agile approach to development is particularly interesting from the point of view of interaction design because it incorporates tight iterations and feedback, and collaboration with the customer (e.g. Armitage, 2004; Sharp *et al*, 2006). For example, in eXtreme¹ Programming (XP), each iteration is between one and three weeks, with a product of value being delivered at the end of each iteration. Also, XP stipulates that the customer should be on site with developers. In practice, the customer role is usually taken by a team rather than one person (Martin *et al*, 2009), and integration is far from straightforward (Ferreira *et al*, 2012). Several companies have integrated agile methods with interaction design practices to produce better quality products. [Chapter 12](#) discusses this aspect in more detail.

One of the main proponents for integrating user-centered design and agile development in practice, Jeff Patton, has articulated several patterns of common behaviors for successful agile product development, including the importance of designers being part of the team, using parallel tracks with user research, doing just enough user research, modeling and design up front, buying design time when developers are working on complex engineering stories, and cultivating a user group for continuous user validation. Although they first appeared a while ago, these still represent good and practical advice.

Link to see Jeff Patton's common behaviours for successful agile product development at <http://tinyurl.com/3rfnm2>

These recommendations are echoed in a report from the Nielsen Norman Group (Nielsen Norman Group, 2014) who conducted a study of agile projects, investigating case studies in depth from 16 organizations who care about user experience and who have also embraced an agile approach to development. They make two main recommendations for success: have development and design running in separate tracks (as described in [Figure 12.2](#) in [Chapter 12](#)) and maintain a coherent vision of the interface architecture (also see Kollmann *et al*, 2009).

Assignment

Nowadays, timepieces (such as clocks, wristwatches, etc.) have a variety of functions. They not only tell the time and date but they can speak to you, remind you when it's time to do something, and provide a light in the dark, among other things. Mostly, the interface for these devices, however, shows the time in one of two basic ways: as a digital number such as 23:40 or through an analog display with two or three hands – one to represent the hour, one for the minutes, and one for the seconds.

In this assignment, we want you to design an innovative timepiece for your own use. This could be in the form of a wristwatch, a mantelpiece clock, an electronic clock, or any other kind of timepiece you fancy. Your goal is to be inventive and exploratory. We have broken this assignment down into the following steps to make it clearer:

- a. Think about the interactive product you are designing: what do you want it to do for you? Find three to five potential users and ask them what they would want. Write a list of requirements for the clock, together with some usability criteria and user experience criteria based on the definitions in [Chapter 1](#).
- b. Look around for similar devices and seek out other sources of inspiration that you might find helpful. Make a note of any findings that are interesting, useful, or insightful.
- c. Sketch out some initial designs for the timepiece. Try to develop at least two distinct alternatives that both meet your set of requirements.
- d. Evaluate the two designs, using your usability criteria and by role playing an interaction with your sketches. Involve potential users in the evaluation, if possible. Does it do what you want? Is the time or other information being displayed always clear? Design is iterative, so you may want to return to

earlier elements of the process before you choose one of your alternatives.

Take a Quickvote on Chapter 9:
www.id-book.com/quickvotes/chapter9

Summary

In this chapter, we have looked at user-centered design and the process of interaction design, i.e. what is user-centered design, what activities are required in order to design an interactive product, and how are these activities related. A simple interaction design lifecycle model consisting of four activities was introduced and issues surrounding the involvement and identification of users, generating alternative designs, evaluating designs, and integrating user-centered concerns with other lifecycles were discussed.

Key points

- The interaction design process consists of four basic activities: establishing requirements, designing alternatives that meet those requirements, prototyping the designs so that they can be communicated and assessed, and evaluating them.
- User-centered design rests on three principles: early focus on users and tasks, empirical measurement, and iterative design. These principles are also key for interaction design.
- Involving users in the design process helps with expectation management and feelings of ownership, but how and when to involve users is a matter of dispute.
- Before you can begin to establish requirements, you must understand who the users are and what their goals are in using the product.
- Looking at others' designs and involving other people in design provides useful inspiration and encourages designers to consider alternative design solutions, which is key to effective design.
- Usability criteria, technical feasibility, and users' feedback on prototypes can all be used to choose among alternatives.

- Prototyping is a useful technique for facilitating user feedback on designs at all stages.
- Integrating interaction design activities with other lifecycle models requires careful planning.

Further Reading

Greenbaum, J. and Kyng, M. (eds) (1991) *Design at Work: Co-operative design of computer systems*. Lawrence Erlbaum. This classic book is a good collection of papers about the co-design of software systems: both why it is worthwhile and early experiences of how to do it.

Highsmith, J. (2002) *Agile Software Development Ecosystems*. Addison-Wesley. This book introduces the main agile methods and their proponents. Highsmith explains the motivation behind the agile approach to development and extracts some common themes. The book includes some case studies, and how you the reader can go about developing your own agile method that suits your own particular environment.

Kelley, T., with Littman, J. (2004) *The Art of Innovation*. Profile Books. Tom Kelley is general manager of IDEO. In this book, Kelley explains some of the innovative techniques used at IDEO, but more importantly he talks about the culture and philosophy underlying IDEO's success. There are some useful practical hints in here as well as an informative story about building and maintaining a successful design company.

Nielsen, J. (1993) *Usability Engineering*. Morgan Kaufmann. This is a seminal book on usability engineering. If you want to find out more about the philosophy, intent, history, or pragmatics of usability engineering, then this is the main text.

Seffah, A., Gulliksen, J. and Desmarais, M. C. (2005) *Human-Centered Software Engineering*. Springer. This book is an edited collection of papers focusing on issues relating to the integration of

user-centered design and software engineering. It is split into five parts: introduction; requirements, scenarios, and use cases; principles, myths, and challenges; user-centered design, unified, and agile processes; and user-centered design knowledge and UI design patterns. If you want to pursue the thorny issue of how to bring these two communities together, then this is a good starting point.

Sommerville, I. (2010) *Software Engineering* (9th edn). Addison-Wesley. If you are interested in pursuing the software engineering aspects of the lifecycle models section, then this book provides a useful overview of the main models and their purpose.